

9.1 The Principal Sources of Optimization

A compiler optimization must preserve the **semantics** of the original program. Except in very special circumstances, once a programmer chooses and implements a particular algorithm, the compiler cannot understand enough about the program to replace it with a substantially different and more efficient algorithm.

A compiler knows only how to apply **relatively low-level semantic transformations**, using general facts such as:

- Algebraic identities like $i + 0 = i$.
- Program semantics, such as the fact that performing the same operation on the same values yields the same result.

9.1.1 Causes of Redundancy

There are many **redundant operations** in a typical program.

- **Source-level redundancy:**

Sometimes, redundancy exists in the program source itself. A programmer may find it more direct and convenient to recalculate a result multiple times, expecting the compiler to recognize that only one calculation is necessary.

Redundancy from high-level languages:

More often, redundancy is a **side effect of using high-level languages**. In most programming languages (except C or C++, where pointer arithmetic is allowed), programmers have no choice but to refer to elements of an array or fields in a structure through expressions like:

$A[i][j]$
 $X \rightarrow f1$

As a program is compiled, each of these high-level data-structure accesses expands into a number of **low-level arithmetic operations**, such as the computation of the location of the (i, j) th element of a matrix A .

Shared low-level operations:

Accesses to the same data structure often share many common low-level operations. Programmers are not aware of these low-level details and **cannot remove these redundancies themselves**.

Software engineering perspective:

It is preferable that programmers continue to access data elements by their **high-level names**, because:

1. Programs are easier to write.
2. Programs are easier to understand and maintain.

By having a compiler **eliminate the redundancies**, we get the **best of both worlds**:

The programs are both **efficient** and **easy to maintain**.

9.1.2 A Running Example: Quicksort

- A **quicksort** program fragment is used to illustrate code-improving transformations.

The key fragment involves partitioning an array and swapping elements based on a pivot:

```
i = m - 1;
j = n;
v = a[n];
while (1) {
    do i = i + 1; while (a[i] < v);
    do j = j - 1; while (a[j] > v);
    if (i >= j) break;
    x = a[i]; a[i] = a[j]; a[j] = x; // swap
}
x = a[i]; a[i] = a[n]; a[n] = x;    // final swap
quicksort(m, j);
quicksort(i + 1, n);
```

- **Array access translation:**

High-level array references (e.g., **a[i]**) are expanded into **two low-level operations**:

- Calculate the memory address (e.g., $t = 4 * i$ if integers occupy 4 bytes).
- Access the array element (e.g., $x = a[t]$).

This results in a long sequence of three-address statements, such as:

```
t6 = 4 * i
x = a[t6]
```

Flow graph (Fig. 9.3):

Code is split into **basic blocks** (B1-B6).

B1 is the entry block.

B2 and B3 form individual loops, while B2, B3, B4, and B5 form a larger loop.

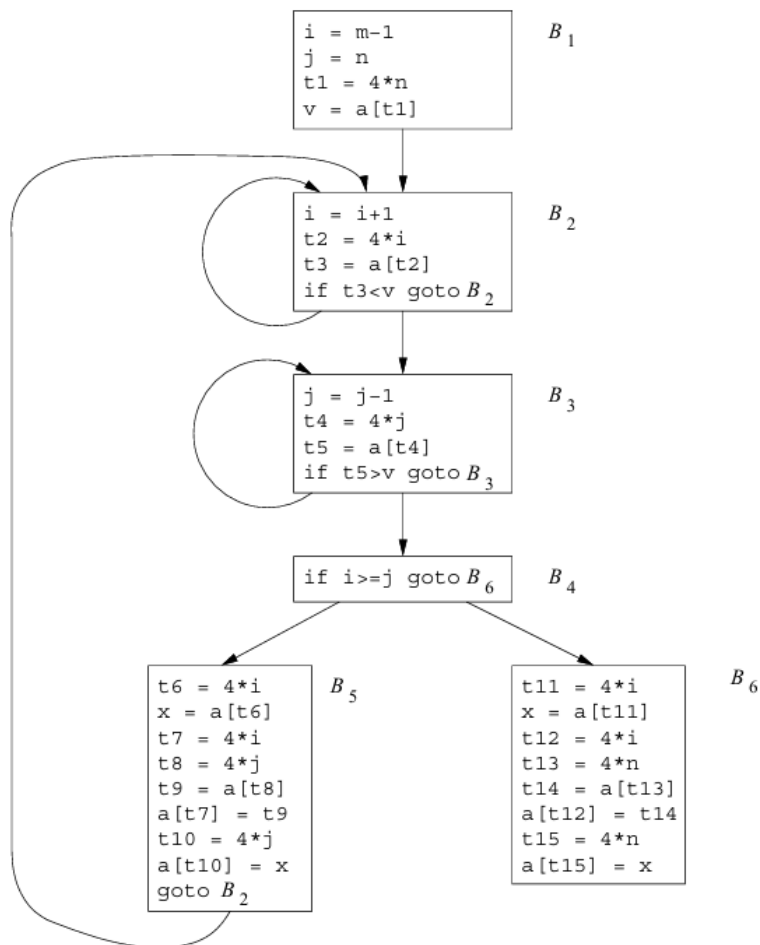


Figure 9.3: Flow graph for the quicksort fragment

9.1.3 Semantics-Preserving Transformations

- Optimizations that do not change the program's result are called **semantics-preserving transformations**.
- Examples include:
 - **Common-Subexpression Elimination (CSE)**: Avoid recomputing the same expression.
 - **Copy Propagation**: Replace variables that are copies of others with the original variable.
 - **Dead-Code Elimination**: Remove computations whose results are never used.
 - **Constant Folding**: Evaluate constant expressions at compile time.

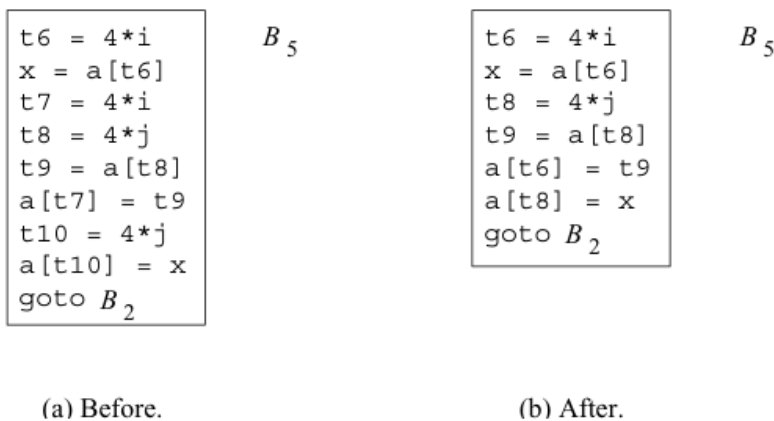


Figure 9.4: Local common-subexpression elimination

9.1.4 Global Common Subexpressions

- A common subexpression is **global** if it appears in **different blocks**, and:
 - The variables in the expression haven't changed.
 - The value of the expression can be safely reused.
- Similarly:
 - $a[t6] = t9$ can be replaced with $a[t6] = t5$ (since $a[j]$ was already read into $t5$).
 - $x = t3$ replaces $x = a[i]$, since $t3$ already holds $a[i]$.
- **Not all subexpressions are safe to reuse:**
E.g., $a[t1]$ is not reused between $B1$ and $B6$ because a might be modified in $B5$.

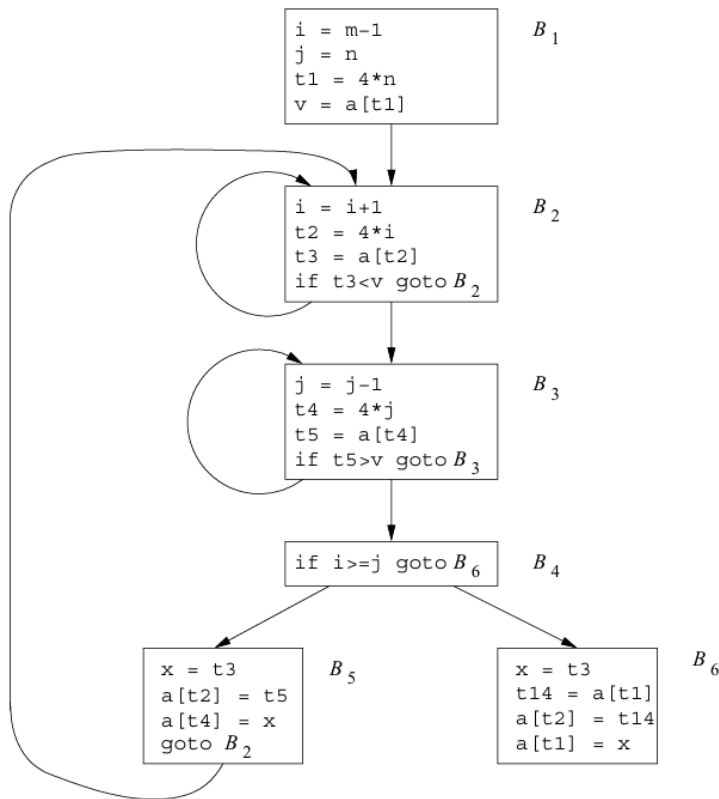


Figure 9.5: B_5 and B_6 after common-subexpression elimination

9.1.5 Copy Propagation

A **copy statement** is an assignment of the form:

$u = v$

Copy propagation replaces uses of u with v wherever possible, allowing the removal of unnecessary variables.

9.1.6 Dead-Code Elimination

- A variable is **live** at a point if its value **will be used later**; otherwise, it is **dead**.
- **Dead code** computes values that are never used and can be removed.

Example 9.4:

If `debug` is always `FALSE` when reaching:

if (debug) print ... this statement is **dead** and can be eliminated.

After copy propagation in B5:

```
x = t3
a[t2] = t5
a[t4] = t3
x = t3 is dead (x is never used), resulting in:
```

```
a[t2] = t5
a[t4] = t3
goto B2
```

9.1.7 Code Motion

- Code motion moves loop-invariant computations outside of loops to reduce repeated work.
- Loop-invariant computation:
A computation whose result does not change during the loop.

Example 9.5:

Original code:

while (i <= limit - 2) Here, **limit - 2** is computed every iteration, but it does not change inside the loop.

Optimized code:

```
t = limit - 2
while (i <= t) Now limit - 2 is computed once before entering the loop instead of n+1 times.
```

9.1.8 Induction Variables and Reduction in Strength

1. What Are Induction Variables?

- A variable **x** is an **induction variable** if, on each assignment within a loop, its value increases or decreases by a constant value **c** (positive or negative).
- **Example:**
 - In the quicksort loop (Fig. 9.5), **i** and **t2 (4*i)** are **induction variables**.
 - On every iteration:
 - **i = i + 1**

- $t2 = 4 * i$ (changes in lock-step with i).

2. Why Optimize Induction Variables?

- Induction variables can be computed more efficiently using **incremental operations** (addition or subtraction) rather than recomputing values (like $4 * i$) on each iteration.
- This leads to **strength reduction**:
 - **Strength Reduction**: Replacing an expensive operation (e.g., multiplication) with a cheaper one (e.g., addition).

3. Strength Reduction Example

In **B3** of Fig. 9.5:

$j = j - 1$

$t4 = 4 * j$

Here, $t4$ ($4 * j$) is recomputed every iteration.

Instead, we can do:

$t4 = t4 - 4$

because $t4$ and j decrease in **lock-step**.

4. Initialization Issue

- When first entering **B3**, $t4$ must be initialized correctly (as $4 * j$).
- This initialization is added at the **end of block B1**, where j itself is initialized.

6. Induction-Variable Elimination (Outer Loop)

- After optimizing **inner loops**, we focus on the **outer loop** (B2, B3, B4, B5).
- **Observation**:
 - $t2 = 4 * i$ and $t4 = 4 * j$ always hold true.
 - We can test $t2 \geq t4$ instead of $i \geq j$.
- Once this replacement is made:
 - i and j become **dead variables**.
 - Their assignments can be removed (dead-code elimination).

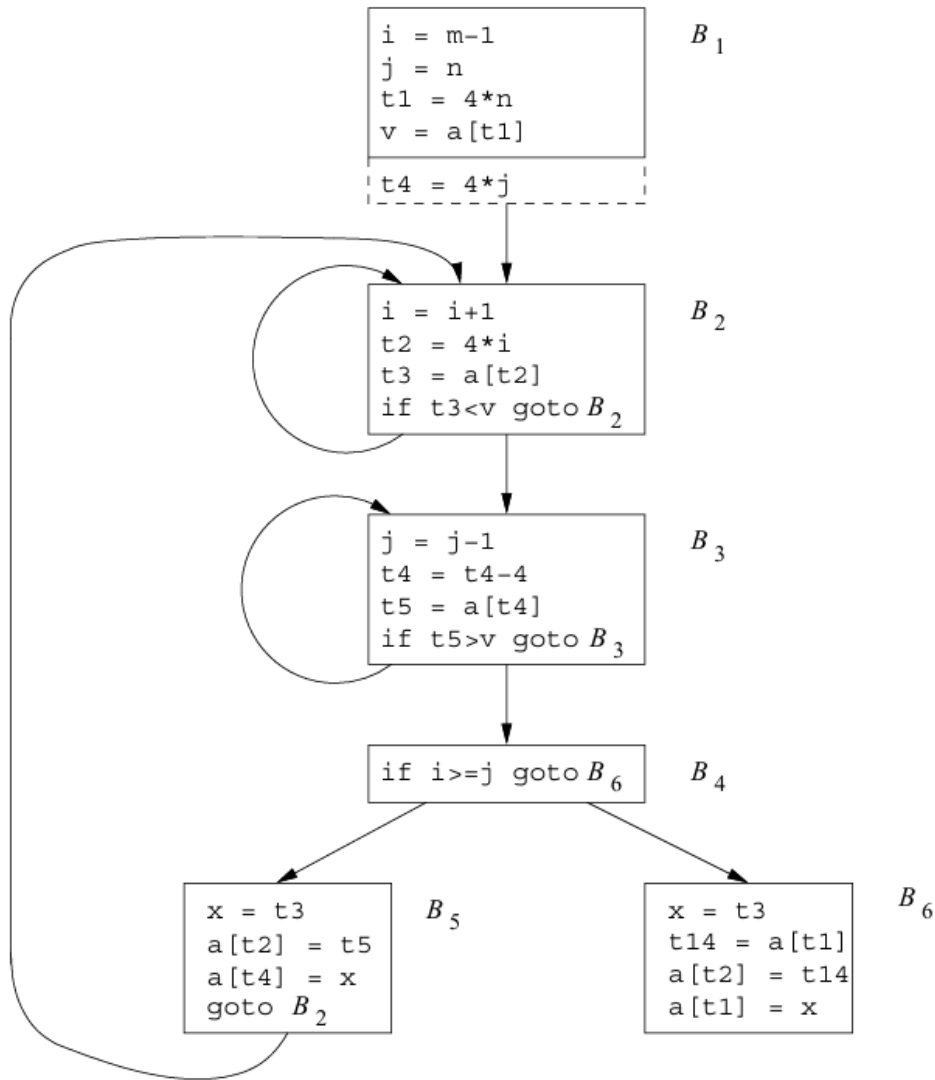


Figure 9.8: Strength reduction applied to $4*j$ in block B_3

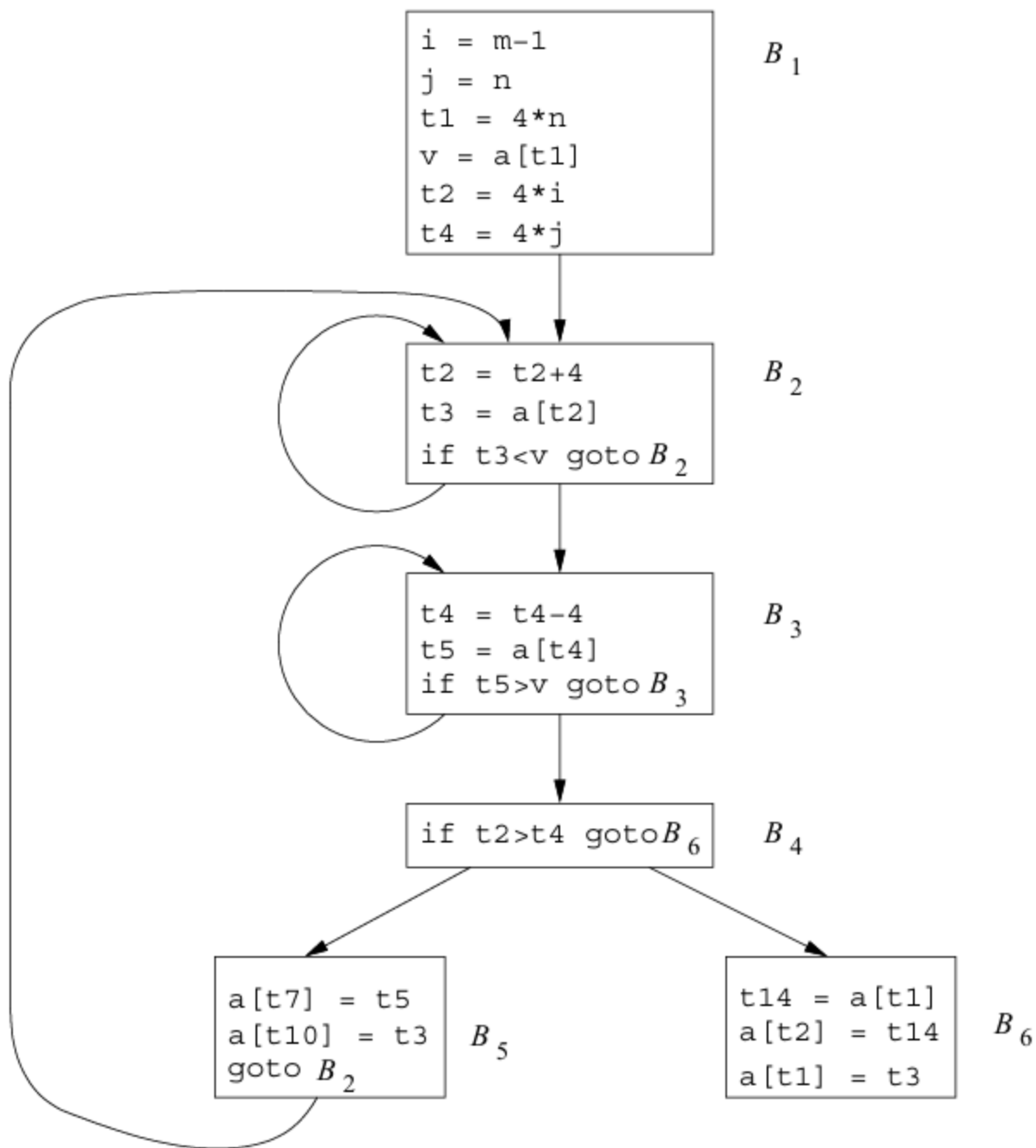


Figure 9.9: Flow graph after induction-variable elimination